

Computer Graphics

Week 2: 2D Graphics (1)

Today's Topics

Pixels, Coordinates, and Colors

- Pixel Coordinates
- Real-number Coordinate Systems
- Aspect Ratio
- Color Models

Shapes

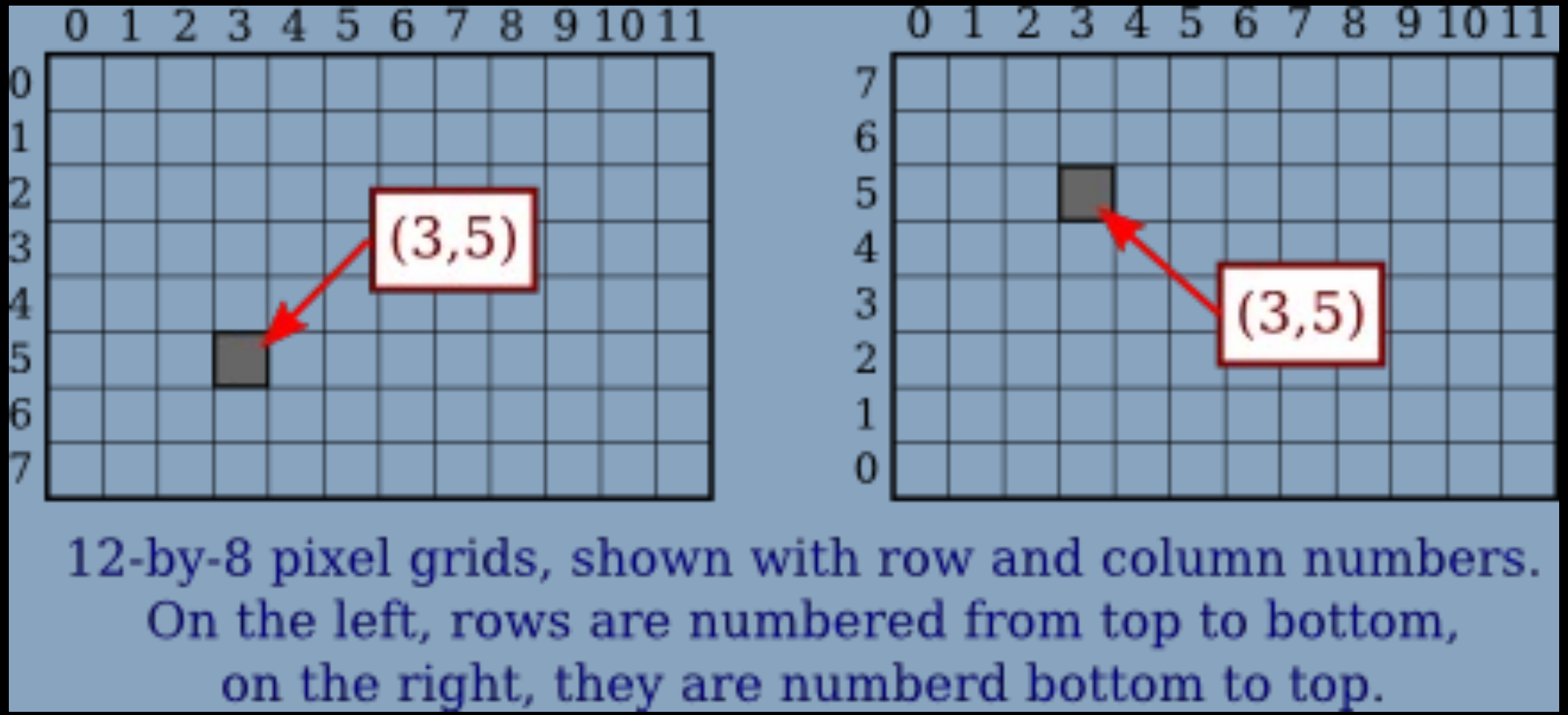
Transforms

- Viewing and Modeling
- Translation
- Rotation
- Combining Transformations
- Scaling
- Matrices and Vectors

Pixels, Coordinates, and Colors

- Pixel Coordinates
- Real-number Coordinate Systems
- Aspect Ratio
- Color Models

Pixel Coordinates

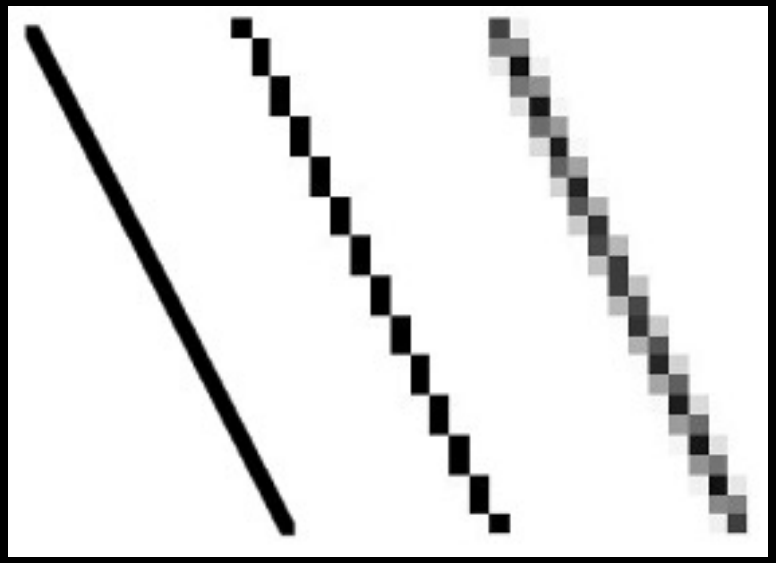


12-by-8 pixel grids, shown with row and column numbers.
On the left, rows are numbered from top to bottom,
on the right, they are numberd bottom to top.

Aliasing

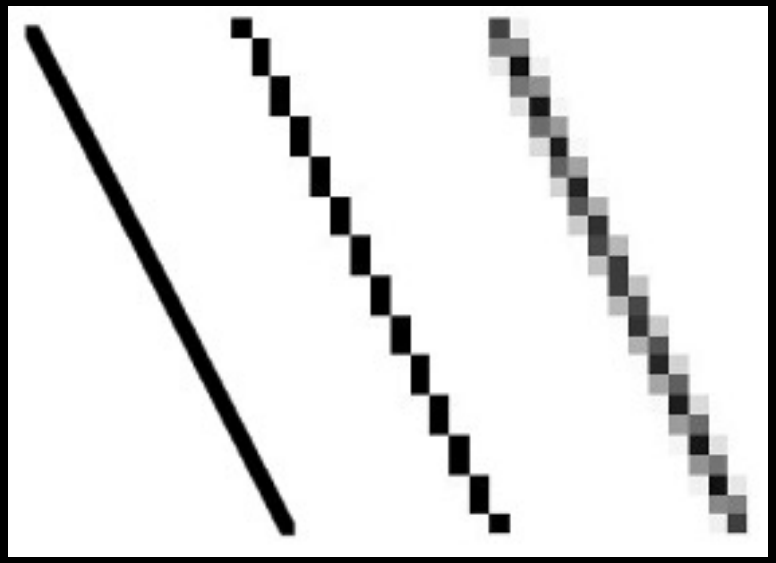
- When you try to draw a line with black and white pixels only, the result is a jagged staircase effect

- When you try to represent the image using pixels, many real-number coordinates will map to the same integer pixel coordinates
  - they can all be considered as different names or "aliases" for the same pixel



Antialiasing

- Techniques to mitigate the effects of aliasing
- When a pixel is only partially covered by a shape, the color of the pixel should be a mixture of the color of the shape and the color of the background
- (left): a geometric line
- (middle): the line is drawn without antialiasing
- (right): the line is drawn using antialiasing

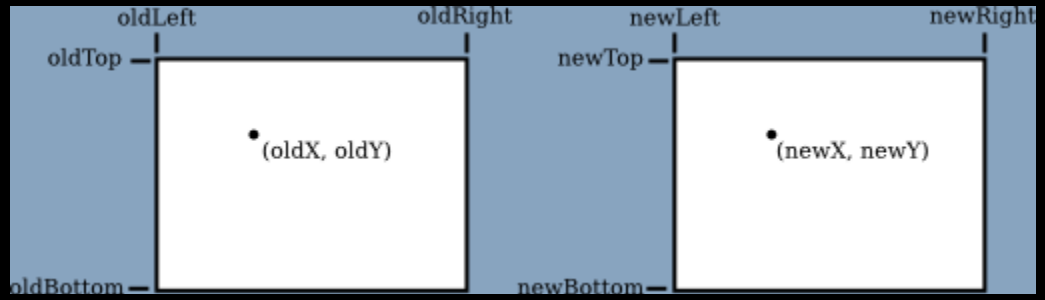


Issues with Mapping Real-number Coordinates to Pixels



- Two lines drawn using HTML Canvas Graphics
  - Top: the line was drawn from (100, 100) to (120, 100)
  - Bottom: the line was drawn from (100.5, 100.5) to (120.5, 100.5)
- In canvas graphics, integer coordinates corresponding to the lines between pixels
- So for the top line, the line is drawn half in one row of pixels and half in another row. The graphics system, which uses antialiasing, rendered the line by coloring both rows of pixels gray.
- Other graphics systems might render the same lines differently.

Real-number Coordinate Systems



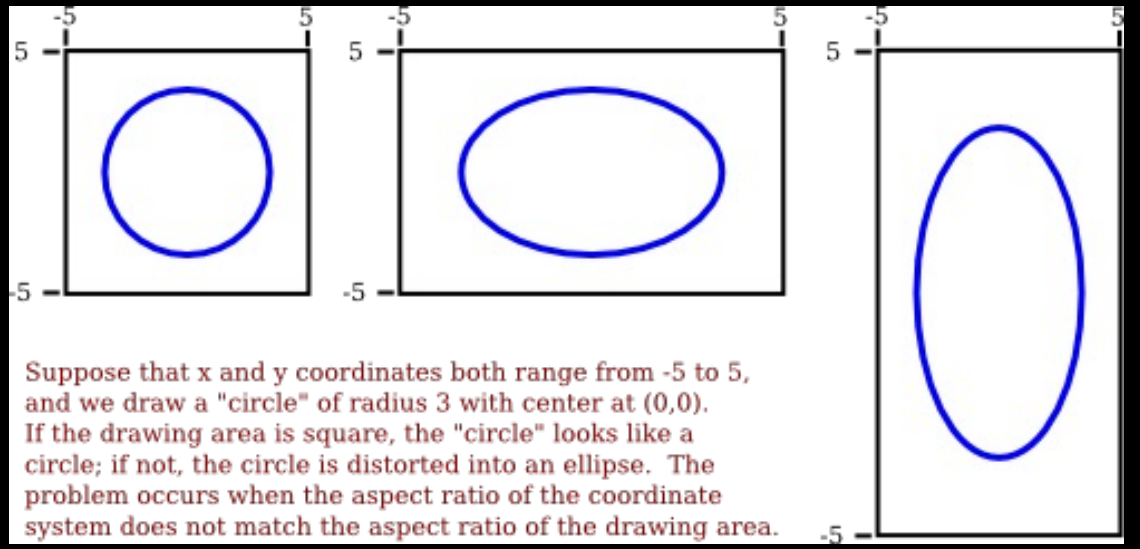
- suppose that we want to transform some real-number coordinate system with limits *left*, *right*, *top*, and *bottom* into pixel coordinates that range from 0 at left to 800 at the right and from 0 at the top 600 at the bottom
  - newX = ((oldX - left) / (right - left)) * 800
  - newY = ((oldY - top) / (bottom - top)) * 600

In practice, though, you won't usually have to do the transformations yourself, since most graphics APIs provide some higher-level way to specify transforms

Aspect Ratio: The ratio of its width to its height

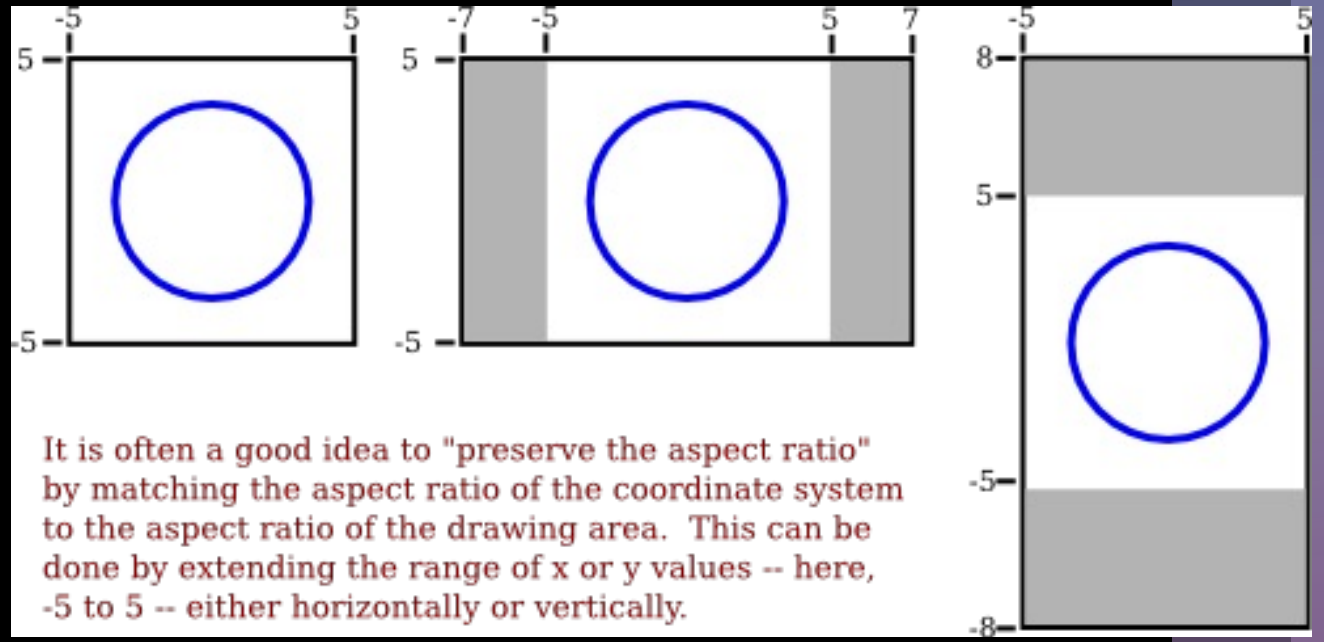
- What is the width of a rectangle with aspect ratio of 5/4, given its height of 600?
- If the horizontal and vertical limits for the coordinate system are left, right, bottom, and top, then the aspect ratio is the absolute value of

  (right - left) / (top - bottom)
- If the coordinate system is used on a rectangle with the same aspect ratio, then when viewed in that rectangle, one unit in the horizontal direction will have the same apparent length as a unit in the vertical direction. If the aspect ratios don't match, then there will be some distortion



Aspect Ratio

- Suppose that you want to use coordinates with limits left, right, bottom, and top, and that you do want to preserve the aspect ratio
- In that case, depending on the shape of the display rectangle, you might have to adjust the values either of left and right or of bottom and top to make the aspect ratios match



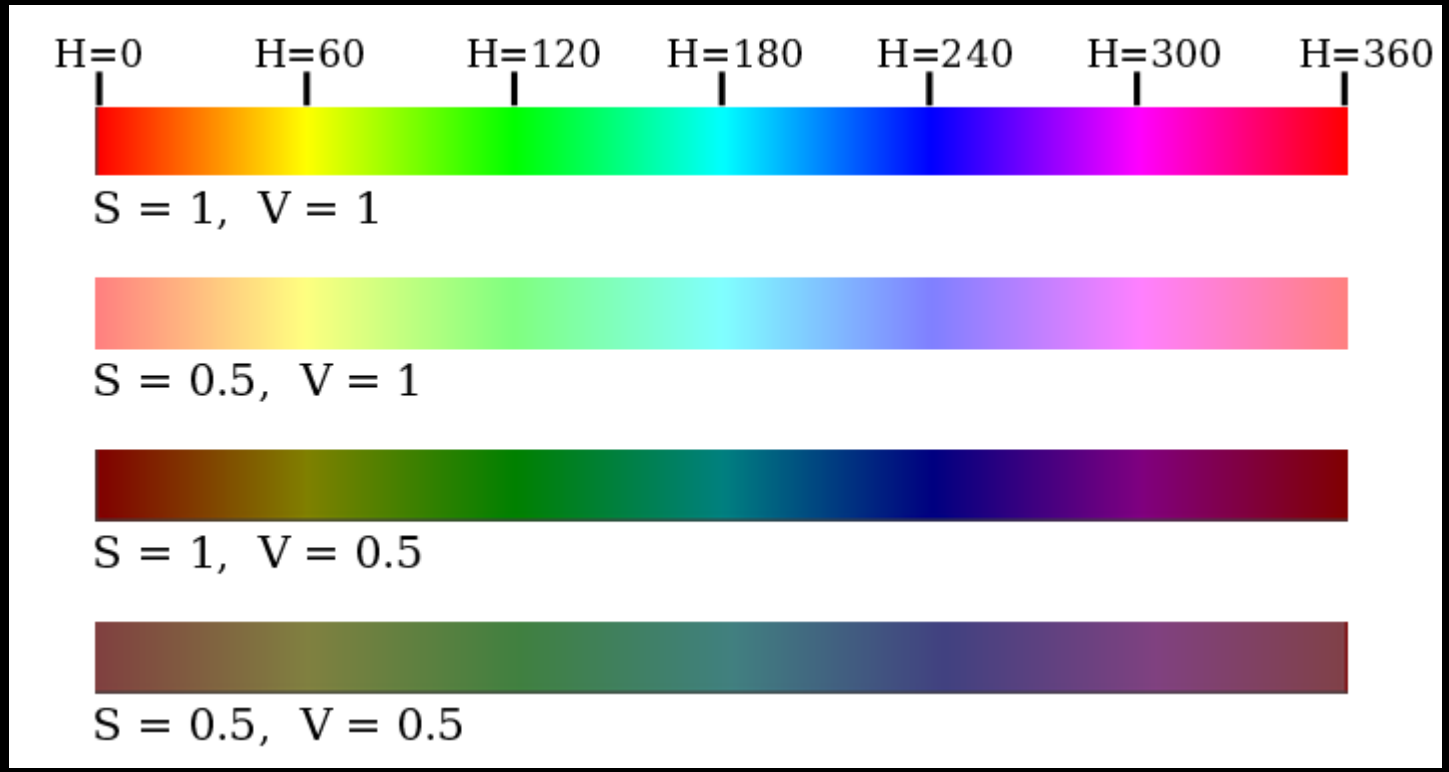
Color Models

- The colors on a computer screen are produced as combinations of red, green, and blue light
- This method of specifying color is called the **RGB color model**
- Three basic colors can produce a reasonably large fraction of the set of perceivable colors, but there are colors that you can see in the world that you will never see on your computer screen
- The range of colors that can be produced by a device such as a computer screen is called the **color gamut** of that device
  - The color gamut of a color printer is noticeably different than the color gamut of a screen

Color Models

- The most common color model for computer graphics is RGB
- The closely related color models HSV and HSL describe the same set of colors as RGB but attempt to do it in a more intuitive way
  - HSV is sometimes called HSB, with the "B" standing for "brightness"
  - HSV and HSB are exactly the same model
  - The "H" in these models stands for "hue," a basic spectral color
    - The value of H is often taken to range from 0 to 360, since the colors can be thought of as arranged around a circle with red at both 0 and 360 degrees
  - The "S" in HSV and HSL stands for "saturation," and is taken to range from 0 to 1
    - A saturation of 1 gives a "pure color," and decreasing the saturation is like adding more gray to the color
  - "V" stands for "value," and "L" stands for "lightness"
    - They determine how bright or dark the color is

The HSV color model



Color Models

- Often, a fourth component is added to color models
- The fourth component is called **alpha**, and color models that use it are referred to by names such as RGBA and HSLA
- Alpha is usually used to represent transparency
- Transparency determines what happens when you draw with one color (the foreground color) on top of another color (the background color)
- Assuming that the alpha component ranges from 0 to 1, the color that you get can be computed as

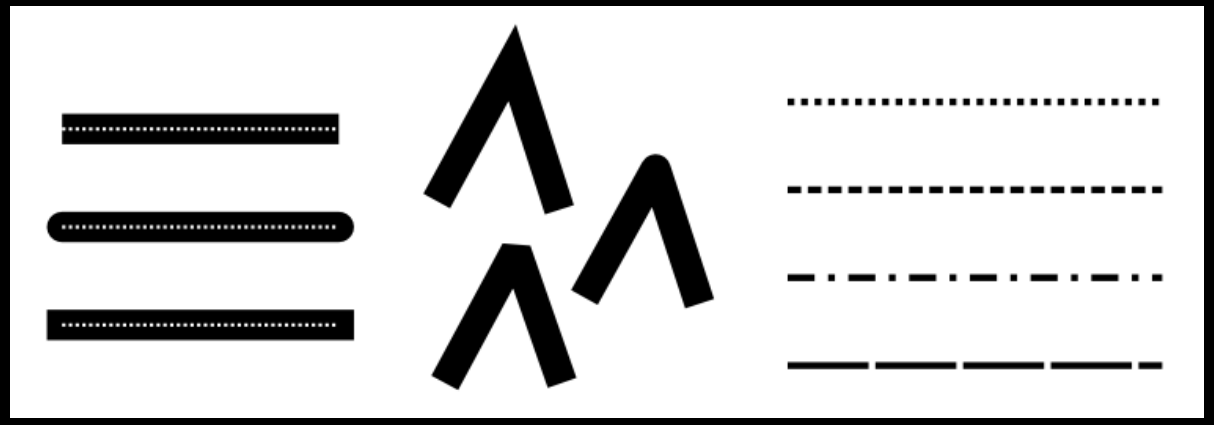
```
new_color = (alpha)*(foreground_color) + (1 - alpha)*(background_color)
```

- This is called **alpha blending**

Shapes

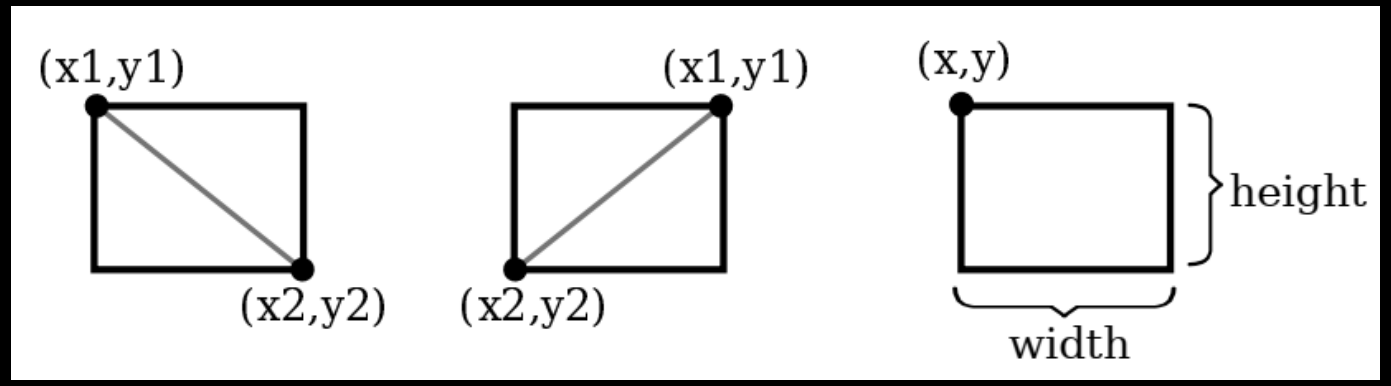
Basic Shapes

- In a graphics API, there will be certain basic shapes that can be drawn with one command
  - In the WebGL API, for example, the only basic shapes are points, lines, and triangles
- A simple one-pixel-wide line segment, without antialiasing, is the most basic shape
  - Lines can have other attributes, or properties, that affect their appearance

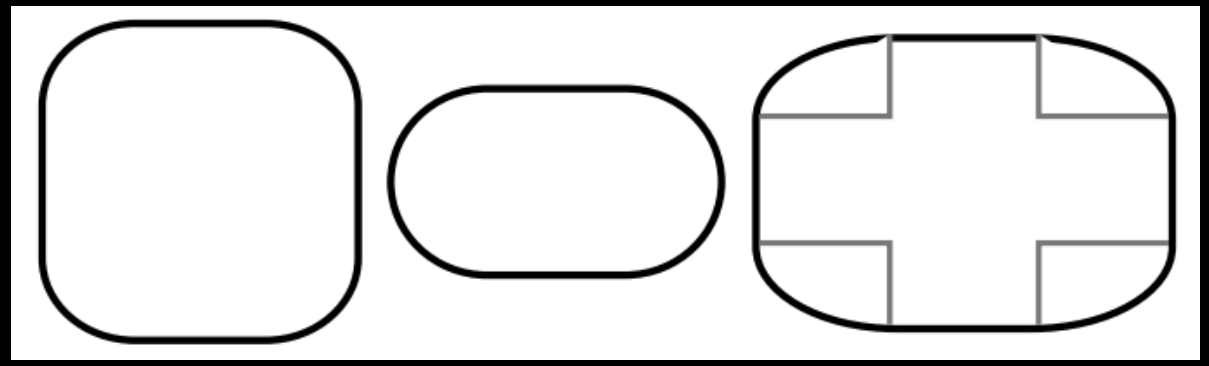


(Left): three wide lines with no cap, a round cap, and a square cap
(Middle): three different styles of line joins: mitered, rounded, and beveled
(Right): four lines with different patters of dots and dashes

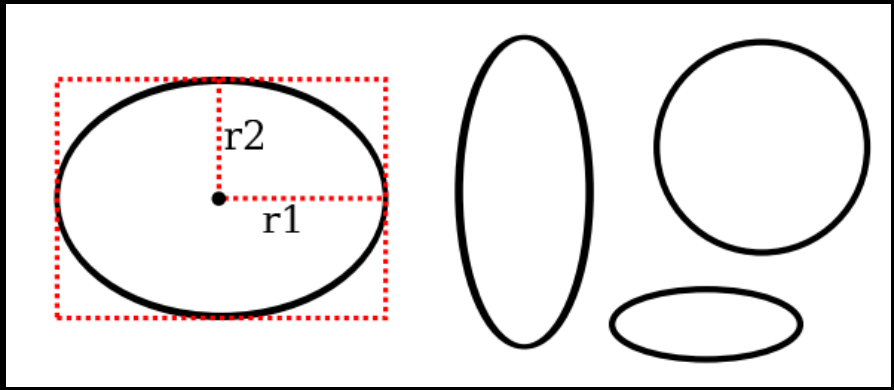
Basic Rectangular Shape



- The basic rectangular shape has sides that are vertical and horizontal
- A common variation on rectangles is to allow rounded corners



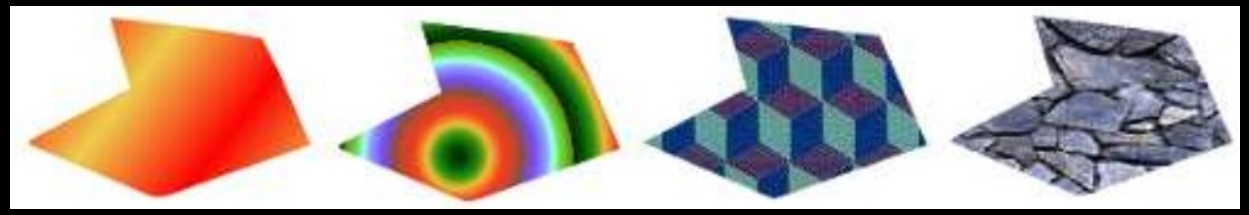
Basic Oval/Ellipse



- If ovals are not available as basic shapes, they can be approximated by drawing a large number of line segments

Stroke and Fill

- Two ways to make a shape visible: stroke and fill
  - Stroking a shape is like dragging a pen along its boundary
  - Filling a shape means coloring all the points that are contained inside that shape
- Shapes can be filled with a color, but other types of fill are possible, including patterns and gradients



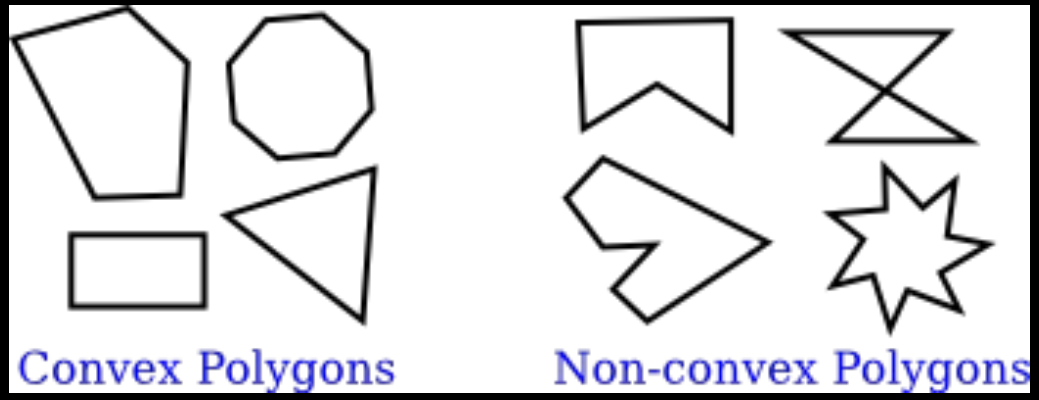
A shape, filled with two gradients and two patterns



The string "Graphics" is filled with a pattern (top) and, filled with a gradient and stroked with solid black (bottom)

Polygons, Curves, and Paths

- A polygon is a closed shape consisting of a sequence of line segments
  - A **convex polygon** has the property that whenever two points are inside or on the polygon, then the entire line segment between those points is also inside or on the polygon



Polygons, Curves, and Paths

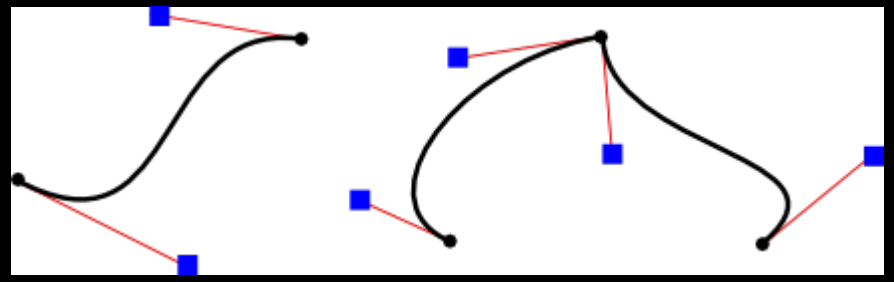
- A polygon is a closed shape consisting of a sequence of line segments
  - A **simple polygon** has the property that the polygon has no self-intersections
  - A **planar polygon** means that all the vertices lie in the same plane
- A **path** is a general shape that can include both line segments and curved segments
  - Suppose that we want a path that represents the triangle with vertices at (100,100), (300,100), and (200, 200):

```
createPath()
moveTo( 100, 100 )
lineTo( 300, 100 )
lineTo( 200, 200 )
closePath()
```

Polygons, Curves, and Paths

- A path can contain other kinds of segments besides lines, e.g. an arc of a circle
- Another type of curve is a Bezier curve, it can be used to create very general curved shapes
- There are two kinds of Bezier curve in common use, cubic Bezier curves and quadratic Bezier curves
- A cubic Bezier curve segment is defined by the two endpoints of the segment together with two control points

Polygons, Curves, and Paths



- Figure above shows three cubic Bezier curve segments
- The two curve segments on the right are connected at an endpoint to form a longer curve
- The endpoints are shown as black dots and the control points as blue squares, with a thin red line connecting each control point to the corresponding endpoint

Transforms

Sub Section

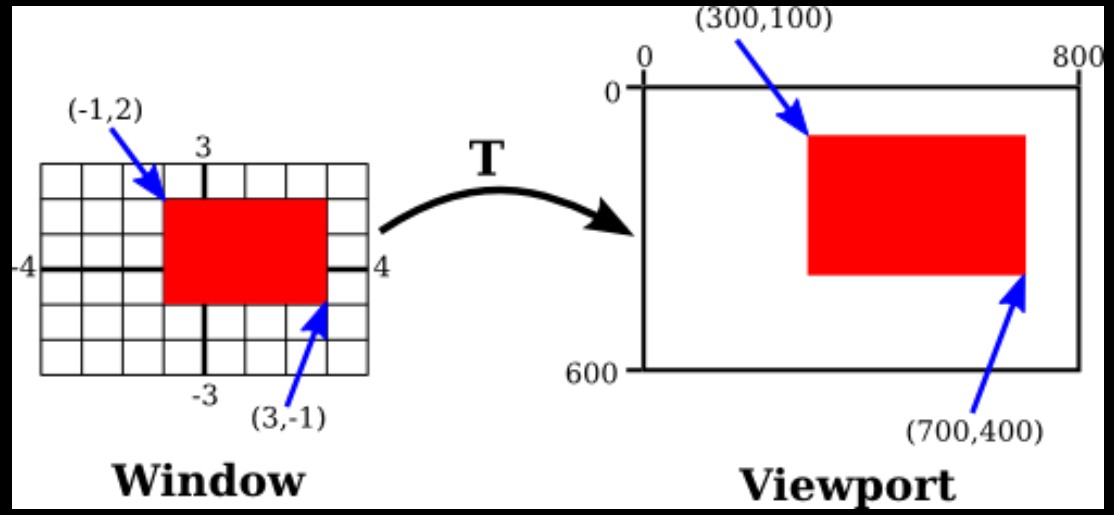
- Viewing and Modeling
- Translation
- Rotation
- Combining Transformations
- Scaling
- Matrices and Vectors

Viewing and Modeling

- **Viewport**: The rectangular area in which the image for 2D or 3D graphics is displayed
  - The coordinates on the viewport are pixel coordinates or device coordinates
- **World coordinates**: The coordinate system in which a scene is defined
- Objects are defined in their own **object coordinate system**.
- **Modeling transformations** are then applied to place objects into the scene; that is, they transform object coordinates to world coordinates

Coordinate Transforms

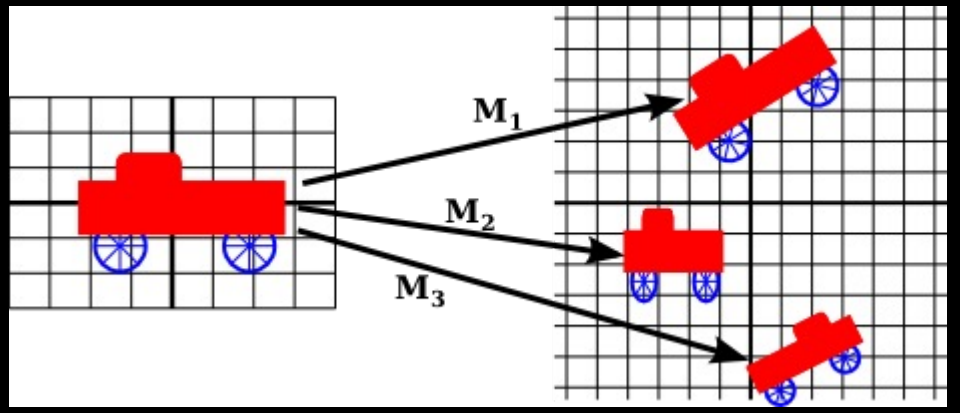
- For 2D graphics, the world lies in a plane
- **View window:** rectangular area in the plane to display in the image
- A **coordinate transform** is used to map the window to the viewport



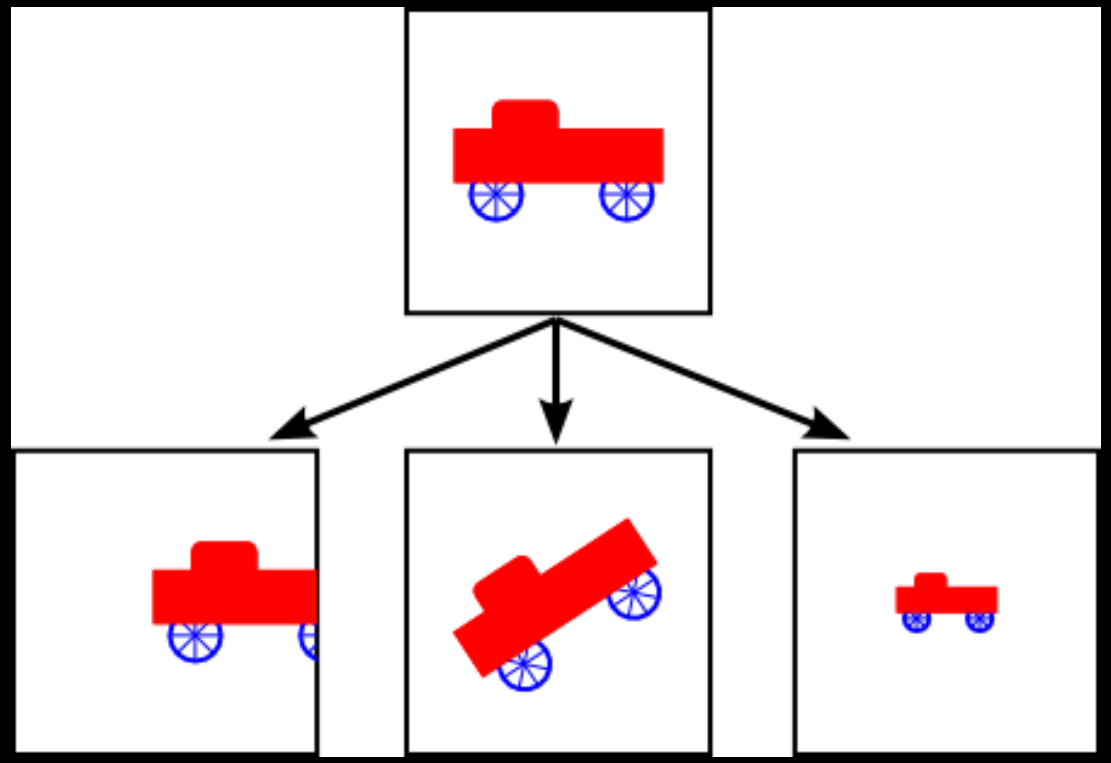
$$T(x,y) = (800*(x+4)/8, 600*(3-y)/6)$$

Modeling Transforms

- The coordinates that we use to define an object are called **object coordinates** for the object
- When we want to place the object into a scene, we need to transform the object coordinates that we used to define the object into the world coordinate system that we are using for the scene
- The transformation that we need is called a **modeling transformation**



Two Ways for The Same Results



Affine Transform

- Geometric transforms are a central concept in computer graphics
- 2D Transformations: if (x,y) represents the coordinates of some point before the transformation is applied, and (x1,y1) are the transformed coordinates

```
x1 = a*x + b*y + e
y1 = c*x + d*y + f
```

- The transform is defined by the six constants a, b, c, d, e, and f.
  - Note that this can be written as a function T, where:

```
T(x,y) = ( a*x + b*y + e, c*x + d*y + f )
```

- A transformation of this form is called an affine transform
  - when it is applied to two parallel lines, the transformed lines will also be parallel
  - if you follow one affine transform by another affine transform, the result is again an affine transform

Translation

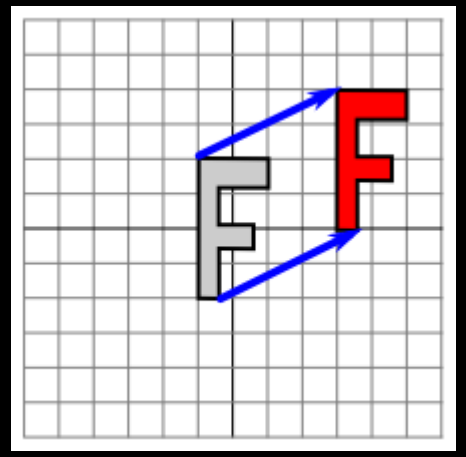
- If (x,y) is the original point and (x1,y1) is the transformed point, then the formula for a translation is

```
x1 = x + e
y1 = y + f
```

  where e is the number of units by which the point is moved horizontally, and f is the amount by which it is moved vertically

- The translation would apply to everything that is drawn after the command is given

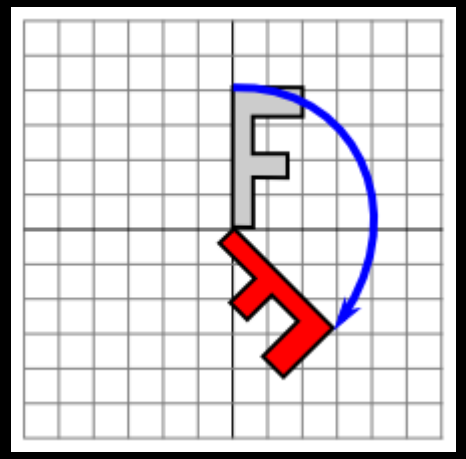
Translation



- Suppose that you draw an "F" using coordinates in which the "F" is centered at (0,0)
- If you say translate(4,2) **before** drawing the "F", then every point of the "F" will be moved horizontally by 4 units and vertically by 2 units before the coordinates are actually used, so that after the translation, the "F" will be centered at (4,2)
- When you give the command translate(e,f), the translation applies to all the drawing that you do after that, not just to the next shape that you draw
- If you apply another transformation after the translation, the second transform will be combined with the translation
  - For example, if you combine translate(4,2) with translate(-1,5), the result is the same as a single translation, translate(3,7)

Also remember that you don't compute coordinate transformations yourself. The computer takes care of applying the transformation to the coordinates. You just need to understand what it does geometrically.

Rotation



What is the angle of rotation in the picture above?

- A rotation transform rotates each point about the origin, (0,0)
- Every point is rotated through the same angle
  - angles can be measured either in degrees or in radians
    - The 2D graphics APIs for Java and JavaScript use radians, but OpenGL and SVG use degrees
- A rotation with a positive angle rotates objects in the direction from the positive x-axis towards the positive y-axis
  - This is counterclockwise in a coordinate system where the y-axis points up
- When rotation through an angle of r radians about the origin is applied to the point (x,y), then the resulting point (x1,y1) is given by

```
x1 = cos(r) * x - sin(r) * y
y1 = sin(r) * x + cos(r) * y
```

- A 2D graphics API would typically have a command rotate(r) to apply a rotation
  - The command is used **before** drawing the objects to which the rotation applies.

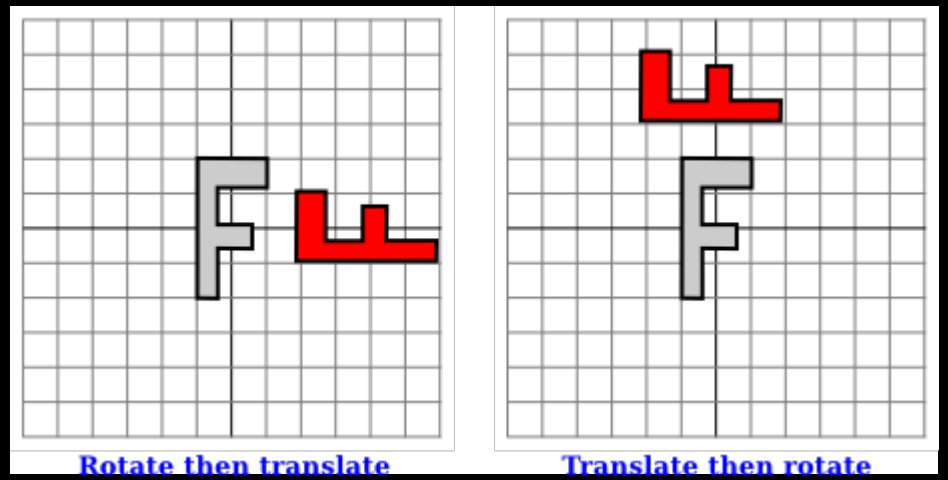
Transform Combination

- Suppose that before drawing some object, you say

  translate(4,0)

  `rotate(90)`

  - Assume that angles are measured in degrees
- Which one is the correct illustration?



Write the code for the transformations to rotate a shape through an angle r about a point (p,q) instead of about the point (0,0)

Scaling

- It can be used to make objects bigger or smaller
- If a point (x,y) is scaled by a factor of a in the x direction and by a factor of d in the y direction, then the resulting point (x1,y1) is given by

```
x1 = a * x
y1 = d * y
```

- Uniform scaling: A scaling transformation in which the scaling factors in all directions are the same
- If you want to scale about a point other than (0,0), you can use a sequence of three transforms, similar to what was done in the case of rotation

Matrices and Vectors

- The transforms that are used in computer graphics can be represented as **matrices**, and the points on which they operate are represented as **vectors**
- If A and B are two N-by-N matrices, then they can be multiplied to give a product matrix C = AB
- If A is an N-by-N matrix, and v is a vector of length N, then v can be multiplied by A to give another vector w = Av
- The function that takes v to Av is a transformation, called a **linear transformation**
  - **Rotation** and **scaling** are linear transformations, why?
  - How about translation?

Matrices and Vectors

- Suppose that A and B are N-by-N matrices and v is a vector of length N
- We can form two different products: A(Bv) and (AB)v
  - These two operations have the same effect
  - We can multiply v by B and then multiply the result by A, or we can multiply the matrices A and B to get the matrix product AB and then multiply v by AB
- If we want to apply a scaling followed by a rotation to the point v = (x,y), we can compute either $R_d(S_{a,b}v)$ or $(R_dS_{a,b})v$
- We could compute the product matrix RdSa,b once and for all, and then apply the combined transform to each point with a single multiplication

multiple transformations can be handled as efficiently as a single transformation

Matrices and Vectors

- To bring translation into this framework, instead of representing a point in 2D as a pair of numbers (x,y), we represent it as the triple of numbers (x,y,1)
- We can then represent rotation, scaling, and translation – and hence any affine transformation – on 2D space as multiplication by a 3-by-3 matrix

$$T_{a,b} = \begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix} \quad S_{a,b} = \begin{pmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad R_{d} = \begin{pmatrix} \cos(d) & -\sin(d) & 0 \\ \sin(d) & \cos(d) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- A sequence of transformations can be combined into a single transformation. The computer only needs to keep track of a single matrix, which we can call the **"current matrix**" or "**current transformation**."

Discussions